

ANLY 500 - Midterm Code Answers

Ralchukwu Mogbogu

Instructions

You will need to complete the portion of the exam before starting the second portion. You will use your answers on this exam to complete the second half. Only knitted files will be graded! Make sure you complete this portion of the exam on your own. Use your notes and class examples to get started with the code.

Abstract

Eventual memory performance is predicted more accurately when a person's judgment of learning (JOL) is delayed until shortly after studying an item than when made immediately after studying the item. According to the transfer-appropriate-monitoring hypothesis, this delayed- JOL effect arises because of the contextual similarity between the cue for the JOL and the cue for the memory test. In a paired-associate learning experiment, delayed JOLs were cued by the stimulus alone or by the stimulus-response pair, and the eventual test was associative recognition of stimulus-response pairs. Recognition of stimulus-response pairs was predicted more accurately when JOLs had been cued by the stimulus alone than when they had been cued by the stimulus-response pair, even though the latter was more similar than the former to the cue for the recognition test. Implications of these results, especially the lack of support for the class of theories emphasizing transfer-appropriate monitoring, are discussed for theories of the accuracy of JOLs.

Method

In these experiments, participants are given English word pairs to learn (i.e. dog-table). They are asked after each word pair: "how confident are you that in about ten minutes from now you will be able to recognize the second of the item when prompted with the first? (0 = definitely won't recognize, 20 = 20% sure, 40 ..., 60 ..., 80 ..., 100 = definitely will recognize). These ratings (JOLs) were given either right after the word pair was presented or after the entire block of word pairs were presented. These ratings were prompted with either just the cue word (i.e. dog, the first word of each pair) or both the words (stimulus-response pairing). After all word-pairs were studied, a 10-minute break was given. Participants were then measured by marking which word-pairs they recognized (0 = not studied, 20 = 20 % sure studied, 40 ..., 60 ..., 80 ..., 100 = definitely studied). You can assume the ratings are continuous.

Dataset:

- JOL group: if they rated their confidence immediately or at the end of each block for how well they would remember the word-pairs.
- Type cue: what type of cue they were using to rate their confidence, either the word-pair or the cue word only.
- Confidence: their average rated confidence to remember word pairs across trial blocks.
- Recognition: their average memory for words across trial blocks.

In this section, the data will be loaded for you from the package. You will change the id = 123456 to your HU id number. This number will generate your data for you in the same time each way, so if you rerun this analysis, you will get the same numbers each time.

```
library(learnSTATS)
library(skimr)
library(mice)
```

```
##
## Attaching package: 'mice'

## The following object is masked from 'package:stats':
##
##   filter

## The following objects are masked from 'package:base':
##
##   cbind, rbind
```

```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
library(ggplot2)
library(corrplot)
```

```
## corrplot 0.92 loaded
```

```
library(MOTE)
```

```
midterm <- midterm_data(idnum = 285166) #CHANGE ME
head(midterm)
```

```
##   JOL_group type_cue   conf1   conf2   conf3   conf4   conf5   conf6
## 1 delayed cue only 49.43136 39.22085 42.08004 47.33668 60.66536 48.14749
## 2 delayed cue only      NA 42.28463 37.41551 53.93699 32.38831 56.28389
## 3 delayed cue only 54.55798 60.03100 54.57673 42.18503 49.82542 46.41933
## 4 delayed cue only 55.80286 56.52724 50.62987 59.26261 57.17489 63.96043
## 5 delayed cue only 71.11040 47.21339 49.50801 34.94682 36.79553 40.43087
## 6 delayed cue only 45.69585 65.44326 46.99567 56.85850 63.88061 53.69672
##   conf7   conf8   conf9  conf10   rec1   rec2   rec3   rec4
```

```
## 1 46.77221 50.91169 63.02401 37.56498 65.44490 66.46937 62.29819 59.95354
## 2 45.08544 60.08692 54.87082 25.48172 60.20844 68.32252 55.14849 63.19267
## 3 33.58502 55.29331 39.04706 68.97581 60.25212 62.07766 56.97343 54.83366
## 4 55.32788 63.73885 49.70083 52.54598 60.27775 59.24452 59.01805 64.96766
## 5 36.31543 67.12227 72.51781 52.76270 69.71816 73.76554 56.48109      NA
## 6 20.74218 38.11565 59.28013 46.07826 55.09285 50.47330 60.41095 64.54275
##      rec5      rec6      rec7      rec8      rec9      rec10
## 1 56.95630 69.93737 55.32542 56.62567 60.31817 60.05987
## 2 62.55615 65.87404 69.63807 68.74171 65.63843 63.75038
## 3 51.20070 86.49010 69.14775 74.49445 67.43084 62.67616
## 4 54.48227 61.46551 58.34279 55.79897 63.41667 67.04844
## 5 51.64496 84.24222 66.46195 61.14387 44.49215 60.50726
## 6 54.94986 49.22027 59.03560 51.97630 60.65529 50.91658
```

Summary:

My ID of **285166** is inputted.

Data screening:

Accuracy:

- Include output and indicate how the data are not accurate.
- Include output to show how you fixed the accuracy errors, and describe what you did.

```
skim(midterm)
```

Table 1: Data summary

Name	midterm
Number of rows	264
Number of columns	22
Column type frequency:	
character	2
numeric	20
Group variables	None

Variable type: character

skim_variable	n_missing	complete_rate	min	max	empty	n_unique	whitespace
JOL_group	0	1.00	7	9	0	2	0
type_cue	7	0.97	8	14	0	2	0

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
conf1	7	0.97	49.27	9.55	21.18	42.84	49.43	55.94	71.11	
conf2	7	0.97	48.34	10.82	16.09	40.96	48.12	55.99	77.21	
conf3	7	0.97	50.24	9.21	27.22	43.80	50.68	55.94	77.48	
conf4	7	0.97	46.56	16.20	-26.97	39.52	48.44	56.25	103.42	
conf5	7	0.97	49.45	10.41	19.44	42.40	49.98	56.22	79.62	
conf6	7	0.97	50.27	10.45	21.98	42.64	50.02	58.10	76.51	
conf7	7	0.97	49.23	10.42	18.84	42.84	49.93	55.95	75.76	
conf8	7	0.97	51.03	9.81	27.23	43.93	51.35	57.64	81.11	
conf9	7	0.97	49.42	11.40	22.15	42.24	49.32	57.55	80.86	
conf10	7	0.97	52.80	39.98	-174.26	42.17	50.28	59.44	256.25	
rec1	7	0.97	60.42	4.68	47.84	57.33	60.44	63.59	72.38	
rec2	7	0.97	59.81	5.39	42.62	56.65	59.71	63.56	75.24	
rec3	7	0.97	60.22	5.34	42.77	56.41	60.26	64.22	73.35	
rec4	7	0.97	59.59	4.92	46.32	56.41	59.30	63.19	71.22	
rec5	7	0.97	59.24	11.08	10.74	55.08	59.89	63.94	107.46	
rec6	7	0.97	60.29	8.79	21.44	56.50	60.04	63.97	87.58	
rec7	7	0.97	60.04	4.80	47.11	56.61	60.10	63.32	76.03	
rec8	7	0.97	59.31	4.93	47.93	55.79	59.11	62.41	75.11	
rec9	7	0.97	59.63	5.90	44.49	55.67	59.68	63.91	78.69	
rec10	7	0.97	60.00	4.63	48.27	57.09	60.46	63.16	72.83	

```

# Creating a copy of the original dataset
copy_midtermDf = midterm

# handling categorical variables
## # Fixing the accuracy errors
copy_midtermDf$JOL_group <- as.factor(copy_midtermDf$JOL_group)
copy_midtermDf$type_cue <- as.factor(copy_midtermDf$type_cue)

# handling continuous variables
## selecting only numerical variables
col_numeric <- sapply(copy_midtermDf, is.numeric)
numerical_values <- copy_midtermDf[, col_numeric]

summary(numerical_values)

```

```

##      conf1      conf2      conf3      conf4
## Min.   :21.18  Min.   :16.09  Min.   :27.22  Min.   : -26.97
## 1st Qu.:42.84  1st Qu.:40.96  1st Qu.:43.80  1st Qu.: 39.52
## Median :49.43  Median :48.12  Median :50.68  Median : 48.44
## Mean   :49.27  Mean   :48.34  Mean   :50.24  Mean   : 46.56
## 3rd Qu.:55.94  3rd Qu.:55.99  3rd Qu.:55.94  3rd Qu.: 56.25
## Max.   :71.11  Max.   :77.21  Max.   :77.48  Max.   :103.42
## NA's   :7      NA's   :7      NA's   :7      NA's   :7
##      conf5      conf6      conf7      conf8
## Min.   :19.44  Min.   :21.98  Min.   :18.84  Min.   :27.23
## 1st Qu.:42.40  1st Qu.:42.64  1st Qu.:42.84  1st Qu.:43.93
## Median :49.98  Median :50.02  Median :49.93  Median :51.35
## Mean   :49.45  Mean   :50.27  Mean   :49.23  Mean   :51.03
## 3rd Qu.:56.22  3rd Qu.:58.10  3rd Qu.:55.95  3rd Qu.:57.64
## Max.   :79.62  Max.   :76.51  Max.   :75.76  Max.   :81.11

```

```
## NA's :7      NA's :7      NA's :7      NA's :7
##      conf9      conf10      rec1      rec2
## Min. :22.15   Min. : -174.26   Min. :47.84   Min. :42.62
## 1st Qu.:42.24   1st Qu.: 42.17   1st Qu.:57.33   1st Qu.:56.65
## Median :49.32   Median : 50.28   Median :60.44   Median :59.71
## Mean :49.42   Mean : 52.80   Mean :60.42   Mean :59.81
## 3rd Qu.:57.55   3rd Qu.: 59.44   3rd Qu.:63.59   3rd Qu.:63.56
## Max. :80.86   Max. : 256.25   Max. :72.38   Max. :75.24
## NA's :7      NA's :7      NA's :7      NA's :7
##      rec3      rec4      rec5      rec6
## Min. :42.77   Min. :46.32   Min. : 10.74   Min. :21.44
## 1st Qu.:56.41   1st Qu.:56.41   1st Qu.: 55.08   1st Qu.:56.50
## Median :60.26   Median :59.30   Median : 59.89   Median :60.04
## Mean :60.22   Mean :59.59   Mean : 59.24   Mean :60.29
## 3rd Qu.:64.22   3rd Qu.:63.19   3rd Qu.: 63.94   3rd Qu.:63.97
## Max. :73.35   Max. :71.22   Max. :107.46   Max. :87.58
## NA's :7      NA's :7      NA's :7      NA's :7
##      rec7      rec8      rec9      rec10
## Min. :47.11   Min. :47.93   Min. :44.49   Min. :48.27
## 1st Qu.:56.61   1st Qu.:55.79   1st Qu.:55.67   1st Qu.:57.09
## Median :60.10   Median :59.11   Median :59.68   Median :60.46
## Mean :60.04   Mean :59.31   Mean :59.63   Mean :60.00
## 3rd Qu.:63.32   3rd Qu.:62.41   3rd Qu.:63.91   3rd Qu.:63.16
## Max. :76.03   Max. :75.11   Max. :78.69   Max. :72.83
## NA's :7      NA's :7      NA's :7      NA's :7
```

```
# summary
print("Summary: Before")
```

```
## [1] "Summary: Before"
```

```
summary(copy_midtermDf)
```

```
##      JOL_group      type_cue      conf1      conf2
## delayed :132      cue only :130   Min. :21.18   Min. :16.09
## immediate:132      stimulus pairs:127 1st Qu.:42.84 1st Qu.:40.96
##      NA's : 7      Median :49.43   Median :48.12
##      Mean :49.27   Mean :48.34
##      3rd Qu.:55.94 3rd Qu.:55.99
##      Max. :71.11   Max. :77.21
##      NA's :7      NA's :7
##      conf3      conf4      conf5      conf6
## Min. :27.22   Min. : -26.97   Min. :19.44   Min. :21.98
## 1st Qu.:43.80   1st Qu.: 39.52   1st Qu.:42.40   1st Qu.:42.64
## Median :50.68   Median : 48.44   Median :49.98   Median :50.02
## Mean :50.24   Mean : 46.56   Mean :49.45   Mean :50.27
## 3rd Qu.:55.94   3rd Qu.: 56.25   3rd Qu.:56.22   3rd Qu.:58.10
## Max. :77.48   Max. :103.42   Max. :79.62   Max. :76.51
## NA's :7      NA's :7      NA's :7      NA's :7
##      conf7      conf8      conf9      conf10
## Min. :18.84   Min. :27.23   Min. :22.15   Min. : -174.26
## 1st Qu.:42.84   1st Qu.:43.93   1st Qu.:42.24   1st Qu.: 42.17
## Median :49.93   Median :51.35   Median :49.32   Median : 50.28
```

```
## Mean :49.23 Mean :51.03 Mean :49.42 Mean : 52.80
## 3rd Qu.:55.95 3rd Qu.:57.64 3rd Qu.:57.55 3rd Qu.: 59.44
## Max. :75.76 Max. :81.11 Max. :80.86 Max. : 256.25
## NA's :7 NA's :7 NA's :7 NA's :7
## rec1 rec2 rec3 rec4
## Min. :47.84 Min. :42.62 Min. :42.77 Min. :46.32
## 1st Qu.:57.33 1st Qu.:56.65 1st Qu.:56.41 1st Qu.:56.41
## Median :60.44 Median :59.71 Median :60.26 Median :59.30
## Mean :60.42 Mean :59.81 Mean :60.22 Mean :59.59
## 3rd Qu.:63.59 3rd Qu.:63.56 3rd Qu.:64.22 3rd Qu.:63.19
## Max. :72.38 Max. :75.24 Max. :73.35 Max. :71.22
## NA's :7 NA's :7 NA's :7 NA's :7
## rec5 rec6 rec7 rec8
## Min. : 10.74 Min. :21.44 Min. :47.11 Min. :47.93
## 1st Qu.: 55.08 1st Qu.:56.50 1st Qu.:56.61 1st Qu.:55.79
## Median : 59.89 Median :60.04 Median :60.10 Median :59.11
## Mean : 59.24 Mean :60.29 Mean :60.04 Mean :59.31
## 3rd Qu.: 63.94 3rd Qu.:63.97 3rd Qu.:63.32 3rd Qu.:62.41
## Max. :107.46 Max. :87.58 Max. :76.03 Max. :75.11
## NA's :7 NA's :7 NA's :7 NA's :7
## rec9 rec10
## Min. :44.49 Min. :48.27
## 1st Qu.:55.67 1st Qu.:57.09
## Median :59.68 Median :60.46
## Mean :59.63 Mean :60.00
## 3rd Qu.:63.91 3rd Qu.:63.16
## Max. :78.69 Max. :72.83
## NA's :7 NA's :7
```

```
## # Fixing the accuracy errors for continuous variables
```

```
copy_midtermDf$conf4[which(copy_midtermDf$conf4 < 0 | copy_midtermDf$conf4 > 100)] = NA
```

```
copy_midtermDf$conf10[which(copy_midtermDf$conf10 < 0 | copy_midtermDf$conf10 > 100)] = NA
```

```
copy_midtermDf$rec5[which(copy_midtermDf$rec5 < 0 | copy_midtermDf$rec5 > 100)] = NA
```

```
print("Summary: After")
```

```
## [1] "Summary: After"
```

```
summary(copy_midtermDf)
```

```
## JOL_group type_cue conf1 conf2
## delayed :132 cue only :130 Min. :21.18 Min. :16.09
## immediate:132 stimulus pairs:127 1st Qu.:42.84 1st Qu.:40.96
## NA's : 7 Median :49.43 Median :48.12
## Mean :49.27 Mean :48.34
## 3rd Qu.:55.94 3rd Qu.:55.99
## Max. :71.11 Max. :77.21
## NA's :7 NA's :7
## conf3 conf4 conf5 conf6
## Min. :27.22 Min. : 1.858 Min. :19.44 Min. :21.98
## 1st Qu.:43.80 1st Qu.:39.820 1st Qu.:42.40 1st Qu.:42.64
```

```

## Median :50.68 Median :48.511 Median :49.98 Median :50.02
## Mean :50.24 Mean :47.111 Mean :49.45 Mean :50.27
## 3rd Qu.:55.94 3rd Qu.:56.247 3rd Qu.:56.22 3rd Qu.:58.10
## Max. :77.48 Max. :91.225 Max. :79.62 Max. :76.51
## NA's :7 NA's :11 NA's :7 NA's :7
## conf7 conf8 conf9 conf10
## Min. :18.84 Min. :27.23 Min. :22.15 Min. : 3.671
## 1st Qu.:42.84 1st Qu.:43.93 1st Qu.:42.24 1st Qu.:42.368
## Median :49.93 Median :51.35 Median :49.32 Median :49.664
## Mean :49.23 Mean :51.03 Mean :49.42 Mean :49.613
## 3rd Qu.:55.95 3rd Qu.:57.64 3rd Qu.:57.55 3rd Qu.:57.441
## Max. :75.76 Max. :81.11 Max. :80.86 Max. :92.268
## NA's :7 NA's :7 NA's :7 NA's :40
## rec1 rec2 rec3 rec4
## Min. :47.84 Min. :42.62 Min. :42.77 Min. :46.32
## 1st Qu.:57.33 1st Qu.:56.65 1st Qu.:56.41 1st Qu.:56.41
## Median :60.44 Median :59.71 Median :60.26 Median :59.30
## Mean :60.42 Mean :59.81 Mean :60.22 Mean :59.59
## 3rd Qu.:63.59 3rd Qu.:63.56 3rd Qu.:64.22 3rd Qu.:63.19
## Max. :72.38 Max. :75.24 Max. :73.35 Max. :71.22
## NA's :7 NA's :7 NA's :7 NA's :7
## rec5 rec6 rec7 rec8
## Min. :10.74 Min. :21.44 Min. :47.11 Min. :47.93
## 1st Qu.:55.07 1st Qu.:56.50 1st Qu.:56.61 1st Qu.:55.79
## Median :59.72 Median :60.04 Median :60.10 Median :59.11
## Mean :58.87 Mean :60.29 Mean :60.04 Mean :59.31
## 3rd Qu.:63.82 3rd Qu.:63.97 3rd Qu.:63.32 3rd Qu.:62.41
## Max. :83.36 Max. :87.58 Max. :76.03 Max. :75.11
## NA's :9 NA's :7 NA's :7 NA's :7
## rec9 rec10
## Min. :44.49 Min. :48.27
## 1st Qu.:55.67 1st Qu.:57.09
## Median :59.68 Median :60.46
## Mean :59.63 Mean :60.00
## 3rd Qu.:63.91 3rd Qu.:63.16
## Max. :78.69 Max. :72.83
## NA's :7 NA's :7

```

Summary: Accuracy Checks and Information explained

The ‘midterm’ dataset’s ‘skimr’ output is comprehensive, containing rich information. Within the dataset are 2 categorical variables and 20 numerical variables, each containing precisely 7 missing values in their numerical columns (‘conf1’ to ‘rec10’). The categorical column ‘type_cue’ also contains 7 missing values, which could indicate the presence of outliers. It may be necessary to take additional measures, such as imputing missing values and handling outliers, to ensure the accuracy of subsequent analyses. The next block tackles the missing values issue and outliers. The ‘skimr’ output shows that 7 values need to be added for type_cue, meaning 257 instead of 264, to match the number of observations present—likewise, all the numerical variables with 7 missing values.

I made a copy of the original dataset named ‘midterm.’ I saved it in an object called ‘copy_midtermDf,’ and I did this to be able to perform further analysis and compare it against the original dataset. I modified the data to ensure the analysis is accurate and efficient. Specifically, I worked on the categorical variables ‘JOL_group’ and ‘type_cue.’ I transformed the categories of ‘JOL_group’ and ‘type_cue’ - such as ‘delayed’ and ‘immediate,’ and ‘cue only’ and ‘stimulus pairs’ - into a more easily understandable ‘factor’ format for the computer to work with. My plans include transforming them to one-hot encoding, a widely used and

straightforward method for further handling categorical variables.

I kept the type as-is 'num' for handling numerical variables because changing the type to an integer with float values would truncate the decimal part, which is essential in the analysis. I checked for the labeling for the columns, and it seems reasonably written and meaningful, denoting confidence (ranging from conf1 to conf10) and recognition (ranging from rec1 to rec10). Upon reviewing the numerical data, I have found some negative values that might have occurred due to errors during data entry or collection. These values could also be outliers, representing extreme observations deviating from the majority of the data. Additionally, I noticed 7 missing values in each numerical variable column, which could imply gaps in the data.

So I adjusted the values to be within the constraint as it explicitly says, 'How confident are you that in about ten minutes from now, you will be able to recognize the second of the item when prompted with the first? (0 = definitely won't recognize, 20 = 20% sure, 40 ..., 60 ..., 80 ..., 100 = definitely will recognize)' I ensured that values below 0 and above 100 would result in an NA value. Therefore, the three affected columns were conf4 with the aftermath of 11 missing values, conf10 with 40 missing values, and rec5 with 9 missing values. Removing some of the missing values is recommended to improve the dataset's accuracy and reliability, and this can be up to 5% of the data. Also, excluding extreme values from the dataset is a good idea, which will help solve any issues and give better results, which I will do in the next blocks.

Missing data:

- Using the 5% rule, exclude all rows that have missing data.
- Create separate datasets for your continuous measures and categorical measures (because everyone can have slightly different, estimate all continuous columns regardless of their missingness for columns).
- Estimate the missing data for your continuous measures.
- Combine your categorical columns back to your newly estimated continuous columns.

```
# output that shows you have missing data
# One way to Check for Missing values
apply(copy_midtermDf ,2,function(x) sum(is.na(x)))
```

```
## JOL_group  type_cue    conf1    conf2    conf3    conf4    conf5    conf6
##          0         7         7         7         7         11         7         7
##   conf7    conf8    conf9    conf10    rec1     rec2     rec3     rec4
##          7         7         7         40         7         7         7         7
##   rec5     rec6     rec7     rec8     rec9     rec10
##          9         7         7         7         7         7
```

```
percentmissing = function(x){sum(is.na(x))/length(x)*100}

missingdata = apply(copy_midtermDf,1,percentmissing) # 1 for rows
table(missingdata)
```

```
## missingdata
##           0 4.54545454545455 9.09090909090909 13.6363636363636
##          134             89             28             12
## 22.7272727272727
##           1
```

```
replaceRow = subset(copy_midtermDf, missingdata <= 5) # 5 percent
missingData1 = apply(replaceRow,1,percentmissing)
table(missingData1)
```



```
## missingData1
##           0 4.54545454545455
##          134                89
```

```
cat_col <- sapply(copy_midtermDf, is.factor)

replace_row <- replaceRow[, !cat_col] # removing factor columns for outliers
dont_row <- replaceRow[, cat_col]

temp_no_miss = mice(replace_row) # perform multiple imputation
```

```
##
## iter imp variable
## 1 1 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 1 2 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 1 3 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 1 4 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 1 5 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 2 1 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 2 2 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 2 3 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 2 4 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 2 5 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 3 1 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 3 2 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 3 3 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 3 4 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 3 5 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 4 1 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 4 2 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 4 3 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 4 4 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 4 5 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 5 1 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 5 2 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 5 3 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 5 4 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
## 5 5 conf1 conf2 conf3 conf4 conf5 conf6 conf7 conf8 conf9 conf10 rec1 rec2 rec3 re
```

```
no_miss = complete(temp_no_miss,1)

#Join the data parts together
# binds the two factor column with the dataset that has no missing values
nomissing_midterm <- cbind(dont_row,no_miss)

print("Before::: fixing the missing values for type_cue\n")
```

```
## [1] "Before::: fixing the missing values for type_cue\n"
```

```
skim(nomissing_midterm)
```

Table 4: Data summary

Name	nomissing_midterm
Number of rows	223
Number of columns	22
Column type frequency:	
factor	2
numeric	20
Group variables	None

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
JOL_group	0	1.00	FALSE	2	del: 119, imm: 104
type_cue	4	0.98	FALSE	2	sti: 115, cue: 104

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
conf1	0	1	49.24	9.72	21.18	42.70	49.65	55.96	71.11	
conf2	0	1	48.44	10.95	16.09	40.93	48.27	56.25	77.21	
conf3	0	1	50.57	9.37	27.22	44.03	51.38	56.16	77.48	
conf4	0	1	47.80	13.68	1.86	40.62	48.57	56.39	91.22	
conf5	0	1	49.67	10.76	19.44	42.65	50.45	56.71	79.62	
conf6	0	1	49.71	10.37	21.98	42.36	49.42	57.42	75.44	
conf7	0	1	49.52	10.48	18.84	42.62	51.15	56.11	75.76	
conf8	0	1	51.18	9.83	27.23	44.02	50.91	57.69	81.11	
conf9	0	1	49.41	11.15	22.15	42.69	49.66	57.14	80.86	
conf10	0	1	49.91	14.88	4.93	42.41	49.62	58.23	92.27	
rec1	0	1	60.66	4.64	47.84	57.50	60.54	63.75	72.38	
rec2	0	1	59.73	5.51	42.62	56.49	59.71	63.55	75.24	
rec3	0	1	60.19	5.31	42.77	56.66	60.20	64.04	73.35	
rec4	0	1	59.66	4.92	46.32	56.52	59.30	63.45	71.22	
rec5	0	1	58.86	10.47	10.74	55.18	59.51	63.88	83.36	
rec6	0	1	60.20	9.09	21.44	56.27	59.93	63.94	87.58	
rec7	0	1	60.00	4.68	47.11	56.76	60.15	63.22	71.82	
rec8	0	1	59.31	4.90	47.93	55.83	59.11	62.41	75.11	
rec9	0	1	59.61	5.97	44.49	55.69	59.61	63.91	78.69	
rec10	0	1	60.19	4.73	48.27	57.35	60.67	63.33	72.83	

```

# Getting the mode value for type-cue
type_cue_mode <- names(which.max(table(nomissing_midterm$type_cue)))

# apply the mode value to all the 7 missing values
nomissing_midterm$type_cue[is.na(nomissing_midterm$type_cue)] <- type_cue_mode

# Check if there are missing values
print("After fixing the missing values for type_cue\n")

```

```
## [1] "After fixing the missing values for type_cue\n"
```

```
skim(nomissing_midterm)
```

Table 7: Data summary

Name	nomissing_midterm
Number of rows	223
Number of columns	22
Column type frequency:	
factor	2
numeric	20
Group variables	None

Variable type: factor

skim_variable	n_missing	complete_rate	ordered	n_unique	top_counts
JOL_group	0	1	FALSE	2	del: 119, imm: 104
type_cue	0	1	FALSE	2	sti: 119, cue: 104

Variable type: numeric

skim_variable	n_missing	complete_rate	mean	sd	p0	p25	p50	p75	p100	hist
conf1	0	1	49.24	9.72	21.18	42.70	49.65	55.96	71.11	
conf2	0	1	48.44	10.95	16.09	40.93	48.27	56.25	77.21	
conf3	0	1	50.57	9.37	27.22	44.03	51.38	56.16	77.48	
conf4	0	1	47.80	13.68	1.86	40.62	48.57	56.39	91.22	
conf5	0	1	49.67	10.76	19.44	42.65	50.45	56.71	79.62	
conf6	0	1	49.71	10.37	21.98	42.36	49.42	57.42	75.44	
conf7	0	1	49.52	10.48	18.84	42.62	51.15	56.11	75.76	
conf8	0	1	51.18	9.83	27.23	44.02	50.91	57.69	81.11	
conf9	0	1	49.41	11.15	22.15	42.69	49.66	57.14	80.86	
conf10	0	1	49.91	14.88	4.93	42.41	49.62	58.23	92.27	
rec1	0	1	60.66	4.64	47.84	57.50	60.54	63.75	72.38	
rec2	0	1	59.73	5.51	42.62	56.49	59.71	63.55	75.24	
rec3	0	1	60.19	5.31	42.77	56.66	60.20	64.04	73.35	
rec4	0	1	59.66	4.92	46.32	56.52	59.30	63.45	71.22	
rec5	0	1	58.86	10.47	10.74	55.18	59.51	63.88	83.36	
rec6	0	1	60.20	9.09	21.44	56.27	59.93	63.94	87.58	
rec7	0	1	60.00	4.68	47.11	56.76	60.15	63.22	71.82	
rec8	0	1	59.31	4.90	47.93	55.83	59.11	62.41	75.11	
rec9	0	1	59.61	5.97	44.49	55.69	59.61	63.91	78.69	
rec10	0	1	60.19	4.73	48.27	57.35	60.67	63.33	72.83	

Summary:

Missing data exclusions, separated into two datasets

During the data cleaning process, I removed rows with missing values using the 5% rule during the data cleaning process, which decreased our dataset from 264 to 223 observations while still retaining the 22 variables. Next, I separated the categorical variables (stored in a dataset called 'dont_row') from the numerical variables (stored in a dataset called 'replace_row') and utilized the MICE package to perform multiple imputations on the numeric values. This process created a dataset called 'nomissing_midterm', which had no missing values.

Missing data estimation, combining back together

However, the type_cue column was affected by the 5% rule, resulting in a decrease of missing values from 7 to 4. To address this issue, I performed a mode imputation on the type_cue variable, replacing the missing values with the mode. Consequently, the final result was a cleaned dataset with no missing values. This cleansed dataset is then saved in the object named '**nomissing_midterm**' which is a combined dataset of the categorical and numerical values using this code: `cbind(dont_row,no_miss)`

Outliers:

- Calculate your Mahalanobis distance scores.
- Include a summary of your mahal scores that are greater than the cutoff.
- How many outliers did you have?
- Exclude all outliers.

```
# Removing the factor variables - the categorical variables

midterm_Outlier <- nomissing_midterm[,-c(1,2)]

# convert group to numeric since it stored as factor

# a
mahal = mahalanobis(midterm_Outlier,
                    colMeans(midterm_Outlier, na.rm=TRUE),
                    cov(midterm_Outlier, use ="pairwise.complete.obs")
                    )

# b
summary(mahal)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  3.994  14.494  19.145  19.910  24.473  44.537
```

```
# c
cutoff = qchisq(1-.001,ncol(midterm_Outlier))
print(cutoff) # 45.314
```

```
## [1] 45.31475
```

```
summary(mahal < cutoff) # no outliers
```

```
##      Mode      TRUE
## logical      223
```

```
# d
noOutliers_midterm = subset(midterm_Outlier, mahal < cutoff)
str(noOutliers_midterm)
```

```
## 'data.frame': 223 obs. of 20 variables:
## $ conf1 : num 49.4 35.4 54.6 55.8 71.1 ...
## $ conf2 : num 39.2 42.3 60 56.5 47.2 ...
## $ conf3 : num 42.1 37.4 54.6 50.6 49.5 ...
## $ conf4 : num 47.3 53.9 42.2 59.3 34.9 ...
## $ conf5 : num 60.7 32.4 49.8 57.2 36.8 ...
## $ conf6 : num 48.1 56.3 46.4 64 40.4 ...
## $ conf7 : num 46.8 45.1 33.6 55.3 36.3 ...
## $ conf8 : num 50.9 60.1 55.3 63.7 67.1 ...
## $ conf9 : num 63 54.9 39 49.7 72.5 ...
## $ conf10: num 37.6 25.5 69 52.5 52.8 ...
## $ rec1 : num 65.4 60.2 60.3 60.3 69.7 ...
## $ rec2 : num 66.5 68.3 62.1 59.2 73.8 ...
## $ rec3 : num 62.3 55.1 57 59 56.5 ...
## $ rec4 : num 60 63.2 54.8 65 64.5 ...
## $ rec5 : num 57 62.6 51.2 54.5 51.6 ...
## $ rec6 : num 69.9 65.9 86.5 61.5 84.2 ...
## $ rec7 : num 55.3 69.6 69.1 58.3 66.5 ...
## $ rec8 : num 56.6 68.7 74.5 55.8 61.1 ...
## $ rec9 : num 60.3 65.6 67.4 63.4 44.5 ...
## $ rec10 : num 60.1 63.8 62.7 67 60.5 ...
```

```
summary(noOutliers_midterm)
```

```
##      conf1      conf2      conf3      conf4
## Min.   :21.18  Min.   :16.09  Min.   :27.22  Min.   : 1.858
## 1st Qu.:42.70  1st Qu.:40.93  1st Qu.:44.03  1st Qu.:40.620
## Median :49.65  Median :48.27  Median :51.38  Median :48.572
## Mean   :49.24  Mean   :48.44  Mean   :50.57  Mean   :47.799
## 3rd Qu.:55.96  3rd Qu.:56.25  3rd Qu.:56.16  3rd Qu.:56.389
## Max.   :71.11  Max.   :77.21  Max.   :77.48  Max.   :91.225
##      conf5      conf6      conf7      conf8
## Min.   :19.44  Min.   :21.98  Min.   :18.84  Min.   :27.23
## 1st Qu.:42.65  1st Qu.:42.36  1st Qu.:42.62  1st Qu.:44.02
## Median :50.45  Median :49.42  Median :51.15  Median :50.91
## Mean   :49.67  Mean   :49.71  Mean   :49.52  Mean   :51.18
## 3rd Qu.:56.71  3rd Qu.:57.42  3rd Qu.:56.11  3rd Qu.:57.69
## Max.   :79.62  Max.   :75.44  Max.   :75.76  Max.   :81.11
##      conf9      conf10      rec1      rec2
## Min.   :22.15  Min.   : 4.934  Min.   :47.84  Min.   :42.62
## 1st Qu.:42.69  1st Qu.:42.413  1st Qu.:57.50  1st Qu.:56.49
## Median :49.66  Median :49.622  Median :60.54  Median :59.71
## Mean   :49.41  Mean   :49.915  Mean   :60.66  Mean   :59.73
## 3rd Qu.:57.14  3rd Qu.:58.230  3rd Qu.:63.75  3rd Qu.:63.55
## Max.   :80.86  Max.   :92.268  Max.   :72.38  Max.   :75.24
##      rec3      rec4      rec5      rec6
## Min.   :42.77  Min.   :46.32  Min.   :10.74  Min.   :21.44
## 1st Qu.:56.66  1st Qu.:56.52  1st Qu.:55.18  1st Qu.:56.27
## Median :60.20  Median :59.30  Median :59.51  Median :59.93
```

```
## Mean      :60.19   Mean      :59.66   Mean      :58.86   Mean      :60.20
## 3rd Qu.   :64.04   3rd Qu.   :63.45   3rd Qu.   :63.88   3rd Qu.   :63.94
## Max.      :73.35   Max.      :71.22   Max.      :83.36   Max.      :87.58
##      rec7      rec8      rec9      rec10
## Min.      :47.11   Min.      :47.93   Min.      :44.49   Min.      :48.27
## 1st Qu.   :56.76   1st Qu.   :55.83   1st Qu.   :55.69   1st Qu.   :57.35
## Median    :60.15   Median    :59.11   Median    :59.61   Median    :60.67
## Mean      :60.00   Mean      :59.31   Mean      :59.61   Mean      :60.19
## 3rd Qu.   :63.22   3rd Qu.   :62.41   3rd Qu.   :63.91   3rd Qu.   :63.33
## Max.      :71.82   Max.      :75.11   Max.      :78.69   Max.      :72.83
```

Summary:

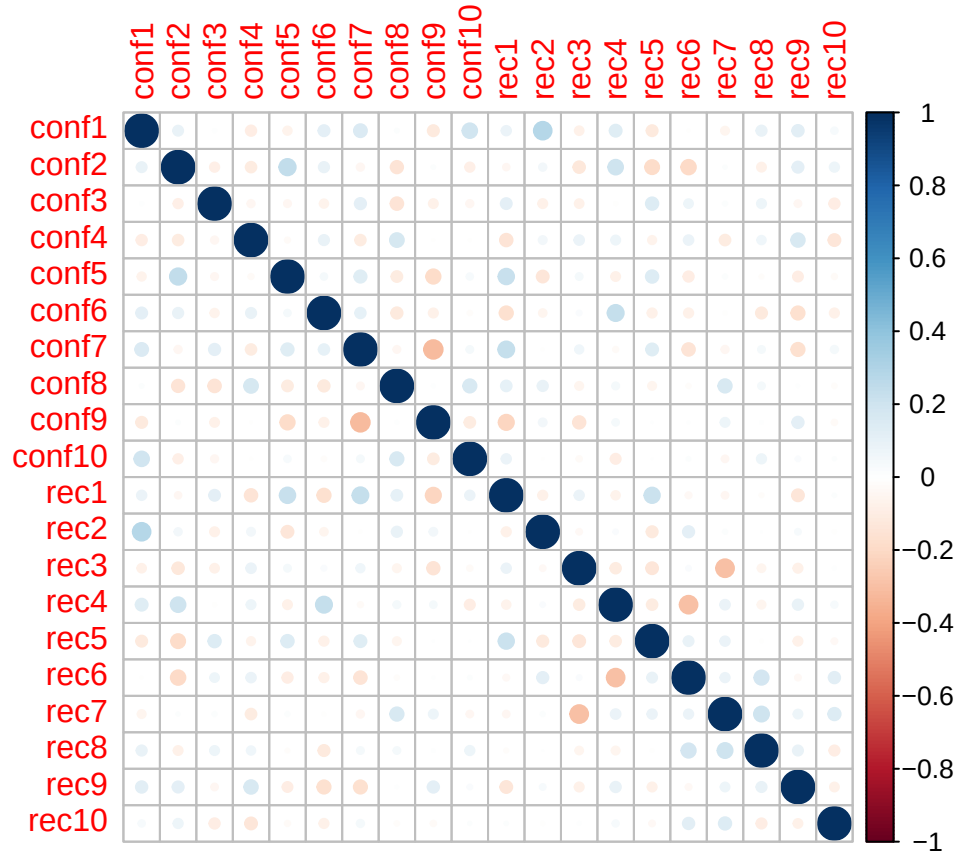
I analyzed a dataset named ‘nomissing_midterm’ and checked for outliers. To do this, I calculated the Mahalanobis distance for each observation based on the numerical values of confidence (conf1 to conf10) and recognition (rec1 to rec10). I then removed the factor variables and combined the Mahalanobis distances. Using the chi-square distribution with degrees of freedom equal to the number of variables, I determined a cutoff value of approximately 45.31. Any observation with a Mahalanobis distance greater than this cutoff was marked as a potential outlier. I found no outliers in this dataset, so I removed the observations that exceeded the cutoff. The resulting dataset, named ‘noOutliers_midterm,’ includes 223 observations with 20 variables, including confidence and recognition ratings. The confidence and recognition ratings range from minimum to maximum across the variables, providing a sense of the distribution and central tendency of the data after outlier removal.

Assumptions:

Additivity:

- Include a `corrplot()` of your continuous measures.

```
corrplot(cor(noOutliers_midterm)) # continuous measures
```



```
symnum(cor(noOutliers_midterm))
```

```
##      cn1 c2 c3 c4 c5 c6 c7 c8 c9 c10 rc1 r2 r3 r4 r5 r6 r7 r8 r9 r10
## conf1 1
## conf2      1
## conf3        1
## conf4          1
## conf5            1
## conf6              1
## conf7                1
## conf8                  1
## conf9                    .      1
## conf10                     1
## rec1                      1
## rec2                       1
## rec3                        1
## rec4                         1
## rec5                          1
## rec6                           1
## rec7                            1
## rec8                             1
## rec9                              1
## rec10                               1
## attr(,"legend")
## [1] 0 ' ' 0.3 '.' 0.6 ',' 0.8 '+' 0.9 '*' 0.95 'B' 1
```

Summary:

The data collected includes various levels of confidence scores ('conf1' to 'conf10') and recognition scores ('rec1' to 'rec10'), but no 'strong' correlation is observed between them.

Linearity:

- Include a picture that shows how you might assess multivariate linearity.

```
random = rchisq(nrow(noOutliers_midterm), 21)
summary(random)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##  5.699  16.618  21.161  21.642  26.383  40.874
```

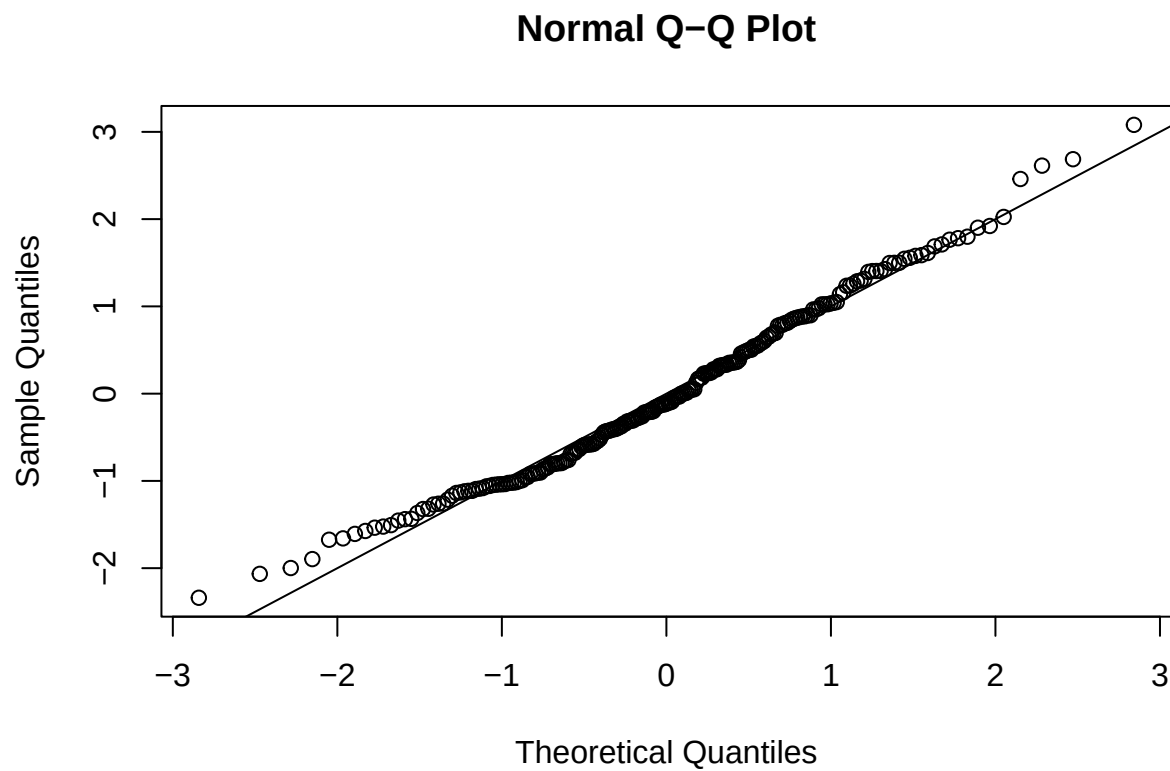
```
m = lm(random~., data = noOutliers_midterm)
summary(m) # fake regression Fake regression information
```

```
##
## Call:
## lm(formula = random ~ ., data = noOutliers_midterm)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -15.4291  -5.3207  -0.7788   4.9065  20.0447
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  37.15712    18.74822   1.982  0.0488 *
## conf1         0.04189     0.05629   0.744  0.4576
## conf2        -0.04399     0.04924  -0.894  0.3726
## conf3         0.02105     0.05315   0.396  0.6925
## conf4         0.01784     0.03821   0.467  0.6410
## conf5         0.03150     0.04940   0.638  0.5244
## conf6         0.04514     0.05158   0.875  0.3826
## conf7        -0.03931     0.05099  -0.771  0.4416
## conf8         0.01192     0.05338   0.223  0.8235
## conf9        -0.09141     0.04637  -1.971  0.0501 .
## conf10       -0.01352     0.03335  -0.405  0.6856
## rec1         -0.02532     0.11553  -0.219  0.8268
## rec2         -0.07652     0.09312  -0.822  0.4122
## rec3         -0.10659     0.09703  -1.099  0.2733
## rec4         -0.11124     0.10829  -1.027  0.3055
## rec5         -0.09408     0.04958  -1.897  0.0592 .
## rec6          0.01480     0.05904   0.251  0.8023
## rec7         -0.09420     0.11448  -0.823  0.4115
## rec8         -0.04187     0.10411  -0.402  0.6879
## rec9          0.15095     0.08661   1.743  0.0829 .
## rec10         0.13786     0.10658   1.293  0.1973
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
```

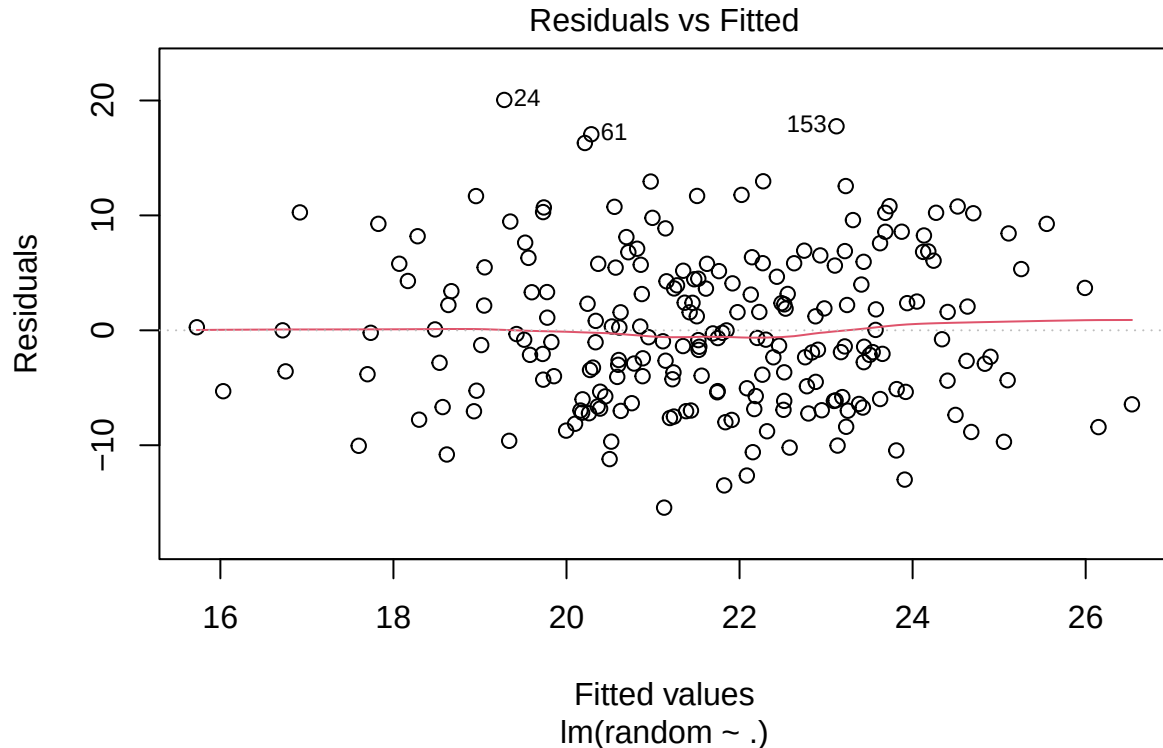


```
## Residual standard error: 6.974 on 202 degrees of freedom
## Multiple R-squared:  0.08278,    Adjusted R-squared:  -0.008036
## F-statistic: 0.9115 on 20 and 202 DF,  p-value: 0.573
```

```
standardized = rstudent(m)
qqnorm(standardized)
abline(0,1)
```



```
plot(m, 1)
```



Summary:

When analyzing multivariate data, we look at all possible combinations for a comprehensive review. If any issues arise, we can use scatter plots to assess whether nonparametric tests should be conducted. To evaluate linearity, I assumed a linear relationship between columns. From this I have met this assumption, as standardized errors are normally distributed around the zero line. Lastly, we have not found any curvilinear pattern or deviation from normal distribution after examining the plot (i.e., the data is cone-shaped or triangle).

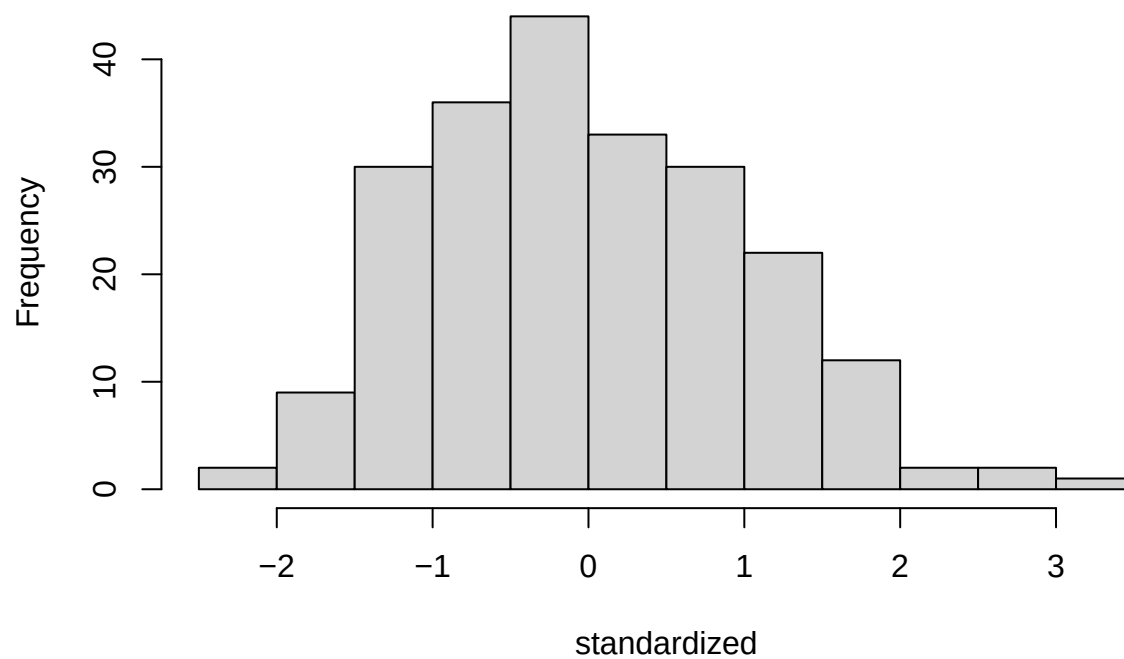
For the **fake regression interpretation**, I fitted the multiple linear regression model to a dataset with the dependent variable 'random' and various predictor variables, including confidence ratings (conf1 to conf10) and recognition ratings (rec1 to rec10). The model showed a non-zero intercept with a statistically significant ($p = 0.0488$). However, only the coefficient for 'conf9' amongst the predictors was statistically significant ($p = 0.0501$), indicating that changes in confidence rating 9 are associated with changes in the dependent variable. However, the overall model has a low R-squared value (0.08278) and a nonsignificant F-statistic ($p = 0.573$), indicating that it lacks strong explanatory power. While the model has identified a significant relationship for 'conf9', it lacks robustness in explaining the variability in the dependent variable 'random' based on the provided predictors.

Normality:

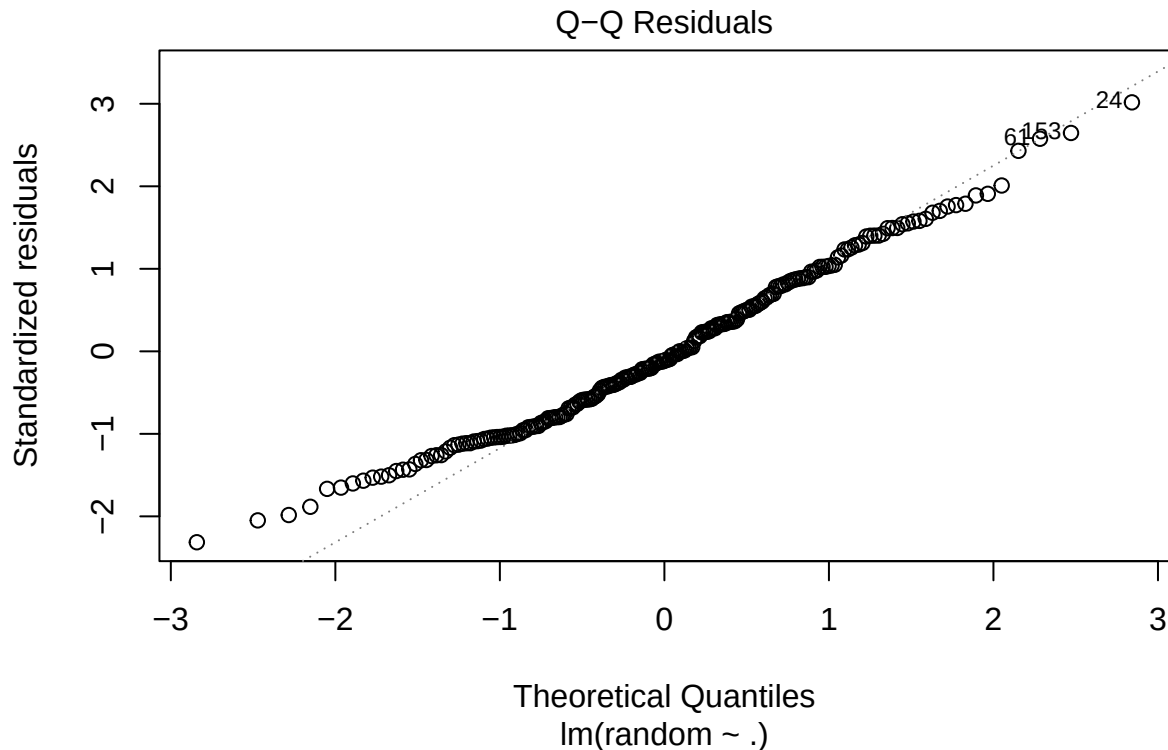
- Include a picture that shows how you might assess multivariate normality.

```
hist(standardized, main= "Histogram of Standardized", xlab = "standardized", breaks=15)
```

Histogram of Standardized



```
plot(m, 2)
```



Summary:

Upon analyzing the histogram, it is evident that the data is not uniformly spread and centered around 0, with some values skewed towards the right. This distribution is not perfectly symmetrical but presents a 'slightly' positively skewed distribution.

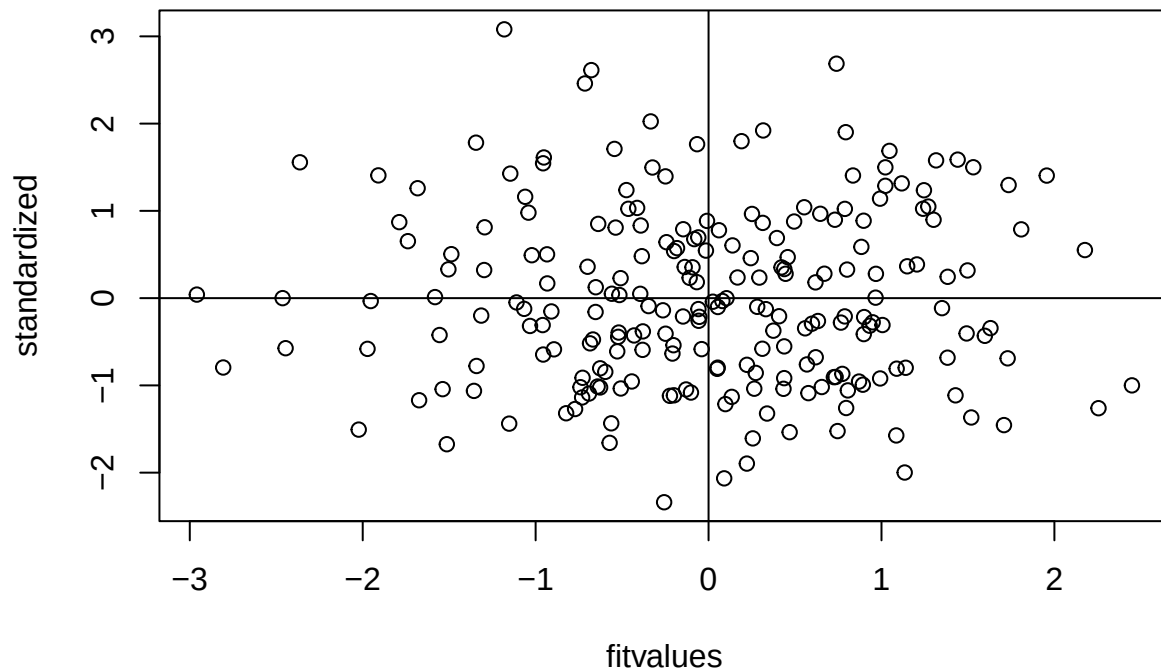
Similarly, upon examining the scatter plot, I observed that over 50 percent of the data stayed close to the line. In contrast, a few data points deviate from the line (especially towards the beginning and end).

Based on these observations, it **meets the assumption of normality**.

Homogeneity/Homoscedasticity:

- Include a picture that shows how you might assess multivariate homogeneity.

```
fitvalues = scale(m$fitted.values)
plot(fitvalues, standardized)
abline(0,0)
abline(v = 0)
```



Summary:

Data is evenly dispersed and spread out due to it meeting the assumption of Homogeneity/Homoscedasticity.

Data exploration:

Create and save average scores for confidence and recognition across trial blocks (two separate columns). Use the following: `apply(dataset[, column #s for confidence or recognition], 1, mean)`. You will use these average scores for the rest of the test.

```
# Using regular expressions to dynamically select columns that start with 'conf' and 'rec.'
conf_col <- grep('^conf', names(nomissing_midterm))

rec_col <- grep('^rec', names(nomissing_midterm))

# Calculate average confidence scores per observation values
nomissing_midterm$avg_confidence <- apply(nomissing_midterm[, conf_col], 1, mean)

# Calculate average recognition scores per observation values
nomissing_midterm$avg_recognition <- apply(nomissing_midterm[, rec_col], 1, mean)

cat("Average scores for Confidence\n")
```

```
## Average scores for Confidence
```

```
summary(nomissing_midterm$avg_confidence)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 38.94   47.34   49.87   49.55   51.89   58.16
```

```
cat("\nAverage scores for Recognition\n")
```

```
##
## Average scores for Recognition
```

```
summary(nomissing_midterm$avg_recognition)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## 54.44   58.73   60.03   59.84   61.14   64.56
```

Create histograms of the confidence and recognition average scores (you can use hist or ggplot2). These charts do not have to be pretty.

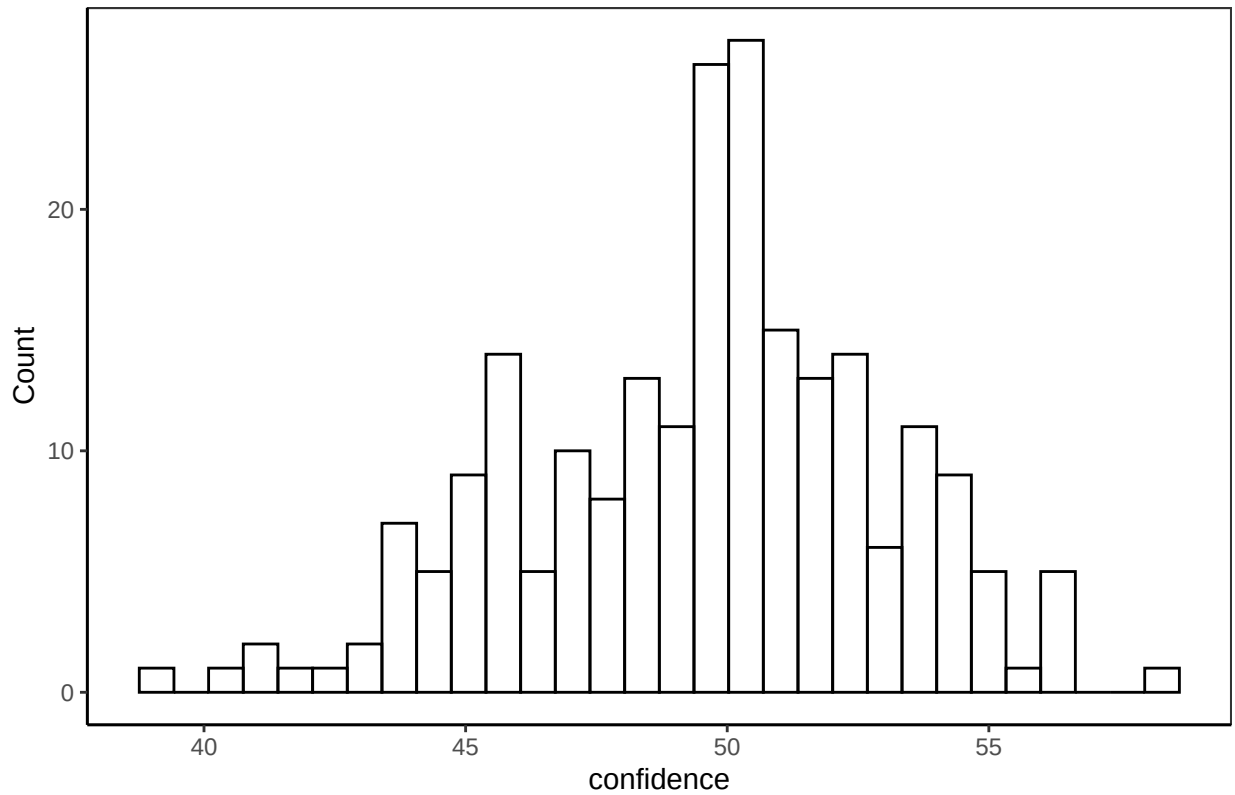
- Do these frequency distributions appear normal?

```
tidyUp = theme(panel.grid.major = element_blank(),
               panel.grid.minor = element_blank(),
               panel.background = element_blank(),
               axis.line= element_line(color = "black"))

confidence_hist = ggplot(data=nomissing_midterm, aes(nomissing_midterm$avg_confidence)) +
  geom_histogram(bins=30,
                col="black",
                fill="white",
                alpha = .2) +
  labs(title="Histogram for average confidence values") +
  labs(x="confidence", y="Count")

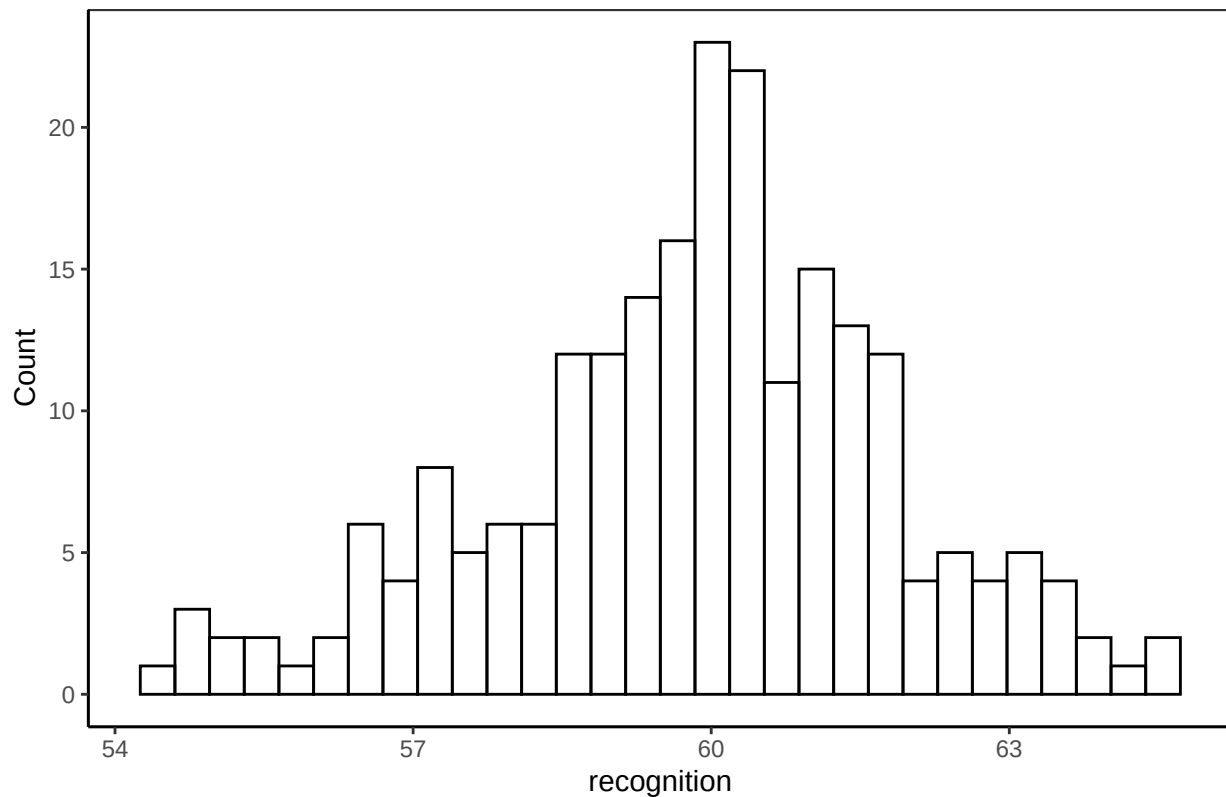
confidence_hist + theme_bw() + tidyUp
```

Histogram for average confidence values



```
recognition_hist = ggplot(data=nomissing_midterm, aes(nomissing_midterm$avg_recognition)) +  
  geom_histogram(bins=30,  
                 col="black",  
                 fill="white",  
                 alpha = .2) +  
  labs(title="Histogram for average recognition values") +  
  labs(x="recognition", y="Count")  
  
recognition_hist + theme_bw() + tidyUp
```

Histogram for average recognition values



Summary:

It meets the assumption of normality.

Print out the means, standard deviations, and length for each condition (JOL-Group and Type Cue combined together) in the study for average recognition only.

```
# Assuming your dataset is named nomissing_midterm
stats_recognition <- nomissing_midterm %>%
  group_by(JOL_group, type_cue) %>%
  summarize(
    means = mean(avg_recognition, na.rm = TRUE),
    standard_deviation = sd(avg_recognition, na.rm = TRUE),
    length = length(avg_recognition),
    .groups = 'drop'
  )

# Stats of each condition (JOL-Group and Type Cue combined together) for avg recognition
print(stats_recognition)
```

```
## # A tibble: 4 x 5
##   JOL_group type_cue      means standard_deviation length
##   <fct>      <fct>      <dbl>          <dbl>      <int>
## 1 delayed   cue only        60.1           2.36         57
## 2 delayed   stimulus pairs    60.1           1.39         62
## 3 immediate cue only        59.3           1.49         47
## 4 immediate stimulus pairs    59.7           2.42         57
```


Use MOTE to calculate the effect size for the difference between recognition scores for the following comparisons:

- Immediate JOL versus Delay JOL for cue only.
- Immediate JOL versus Delay JOL for stimulus pairs.
- What are the sizes of these effects?

```
# Immediate JOL versus Delay JOL for cue only.
cue_only_mote <- subset(nomissing_midterm, type_cue == 'cue only')
delay_for_cue_only <- subset(nomissing_midterm, JOL_group == 'delayed')
immediate_for_cue_only <- subset(nomissing_midterm, JOL_group == 'immediate')

mean_delay <- mean(delay_for_cue_only$avg_recognition)
mean2_immediate <- mean(immediate_for_cue_only$avg_recognition)
sd_delay <- sd(delay_for_cue_only$avg_recognition)
sd_immediate <- sd(immediate_for_cue_only$avg_recognition)

effect_size_cue_only <- d.ind.t(mean_delay, mean2_immediate, sd_delay, sd_immediate, 9, 9, a = .05)

# Immediate JOL versus Delay JOL for stimulus pairs
stimulus_pairs_mote <- subset(nomissing_midterm, type_cue == 'stimulus pairs')
delay_for_stimulus_pairs <- subset(nomissing_midterm, JOL_group == 'delayed')
immediate_for_stimulus_pairs <- subset(nomissing_midterm, JOL_group == 'immediate')

mean_delay_stim <- mean(delay_for_stimulus_pairs$avg_recognition)
mean2_immediate_stim <- mean(immediate_for_stimulus_pairs$avg_recognition)
sd_delay_stim <- sd(delay_for_stimulus_pairs$avg_recognition)
sd_immediate_stim <- sd(immediate_for_stimulus_pairs$avg_recognition)

effect_size_stimulus_pairs <- d.ind.t(mean_delay_stim, mean2_immediate_stim, sd_delay_stim, sd_immediate_stim, 9, 9, a = .05)

# Print the sizes of these effects
cat("MOTE for Immediate vs Delay (cue only):", effect_size_cue_only$d, "\n")
```

```
## MOTE for Immediate vs Delay (cue only): 0.2873459
```

```
cat("MOTE for Immediate vs Delay (stimulus pairs):", effect_size_stimulus_pairs$d, "\n")
```

```
## MOTE for Immediate vs Delay (stimulus pairs): 0.2873459
```

Summary:

Both MOTE sizes appear to be **small** since they are less than 0.5.

Charts:

On all of these questions, be sure to include a coherent label for the X and Y axes. You should change them to be “professional looking” (i.e. Proper Case, explain the variable listed, and could be printed in a journal). The following will be assessed:

- Is it readable?

- Is X-axis labeled appropriately?
- Is Y-axis labeled appropriately?
- Is it the right graph?
- Do the labels look appropriate?
- Are there error bars when appropriate?

Make a bar chart with two independent variables: JOL group, type cue, average recognition. Use `na.omit(dataset)` to exclude any missing data.

```

nomissing_midterm <- na.omit(nomissing_midterm)

barchart_df <- nomissing_midterm %>%
  select(JOL_group, type_cue, avg_recognition)

colnames(barchart_df) = c('df_JOL_group', 'df_type_cue', 'df_value')

#####

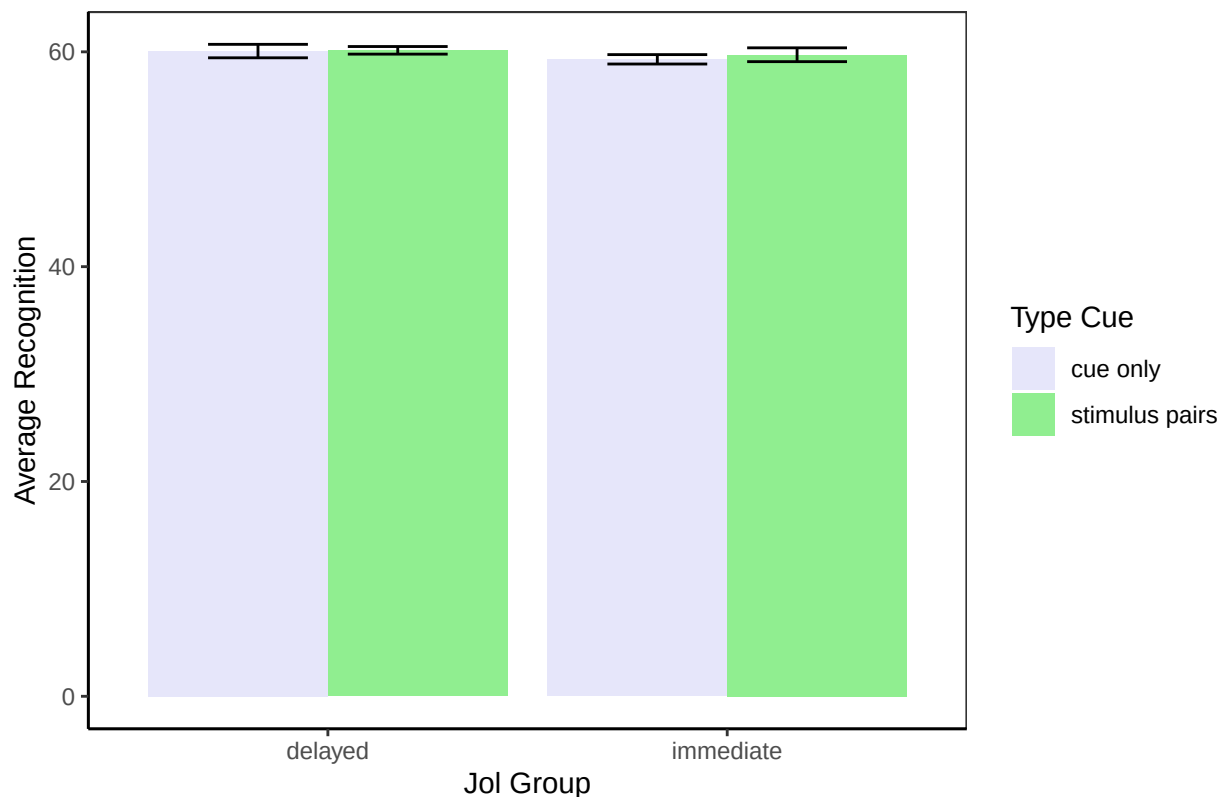
ind_barChart = ggplot(data=barchart_df, aes(x = df_JOL_group, y = df_value, fill = df_type_cue)) +
  stat_summary(fun.y = mean, geom = "bar", position = "dodge") +
  stat_summary(fun.data = mean_cl_normal, geom = "errorbar",
    position = position_dodge(width = 0.70), width = 0.5) +
  xlab("Jol Group") +
  ylab("Average Recognition") +
  scale_fill_manual(name = "Type Cue", values = c('lavender', 'lightgreen')) +
  ggtitle(label = "Bar Chart of 2 Independent Variables - JOL group, Type cue")

#####

ind_barChart + theme_bw() + tidyUp

```

Bar Chart of 2 Independent Variables – JOL group, Type cue



Summary:

The bar chart clearly and concisely represents the average recognition scores for various JOL groups and type cues. The x-axis is labeled appropriately as 'JOL Group,' while the y-axis is labeled 'Average Recognition.' The color coding for different 'type cue' is indicated in the legend. The chart adheres to the specified graph type of a bar chart and includes error bars to represent data variability. The presentation is easily readable, and the black-and-white theme enhances clarity.

Make a repeated measures bar chart: Type of rating (average confidence, average recognition), rating. Hint: create a smaller dataset of just these variables first.

```
# Assuming your dataset is named 'nomissing_midterm'
repeated_measures_df <- nomissing_midterm %>%
  select(JOL_group, type_cue, avg_confidence, avg_recognition)

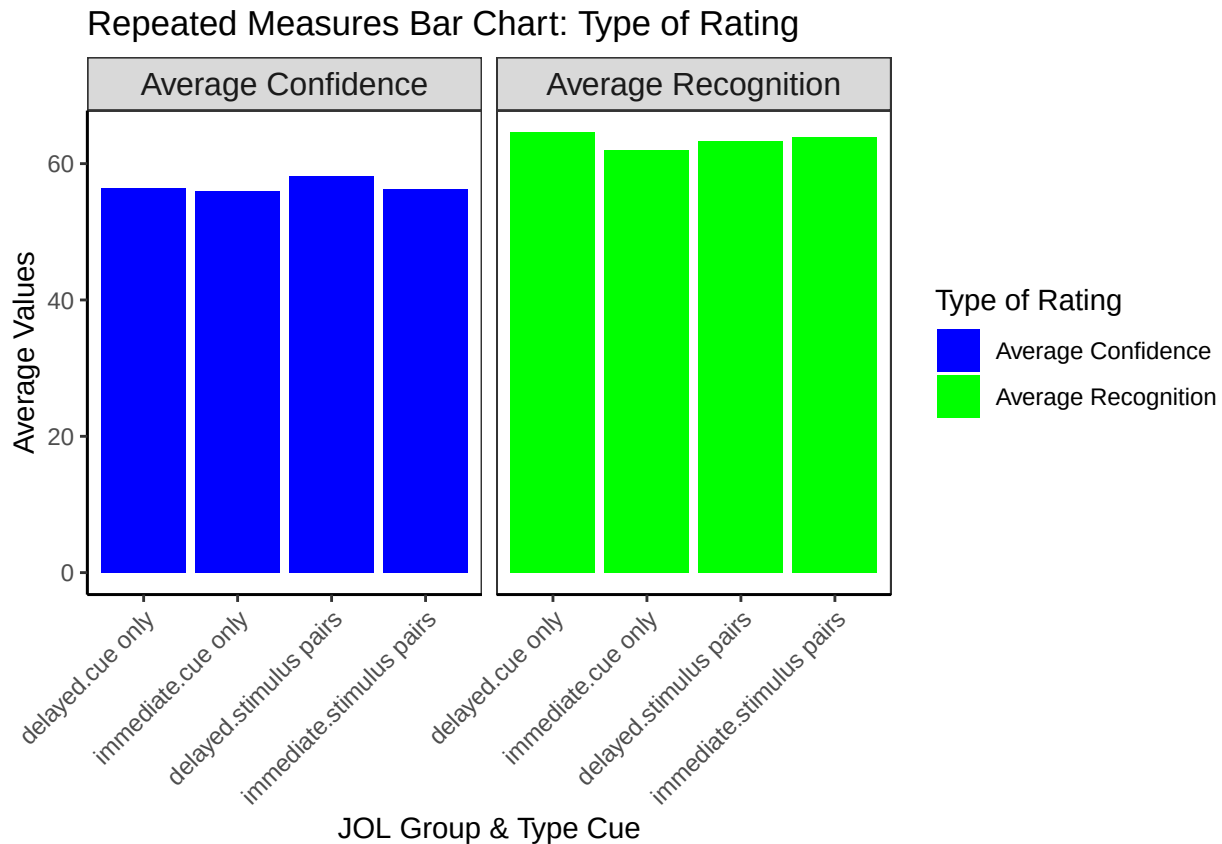
# Rename columns for clarity
repeated_measures_df <- repeated_measures_df %>%
  rename('Average Confidence' = avg_confidence, 'Average Recognition' = avg_recognition)

# Convert the dataset to long format
small_dataset_long <- tidyr::gather(repeated_measures_df, key = "Type of Rating", value = "Average")

# Create a repeated measures bar chart
repeated_measures_chart <- ggplot(small_dataset_long, aes(x = interaction(JOL_group, type_cue), fill = "Type of Rating")) +
  geom_bar(stat = "identity", position = "dodge") +
  facet_grid(. ~ `Type of Rating`, scales = "free_y", space = "free_y") +
  xlab("JOL Group & Type Cue") +
  ylab("Average Values") +
```

```
scale_fill_manual(values = c("Average Confidence" = "blue", "Average Recognition" = "green")) +
ggtitle("Repeated Measures Bar Chart: Type of Rating")

repeated_measures_chart + theme_bw() + theme(axis.text.x = element_text(angle = 45, hjust = 1),
strip.text.x = element_text(size = 12)) + tidyUp
```

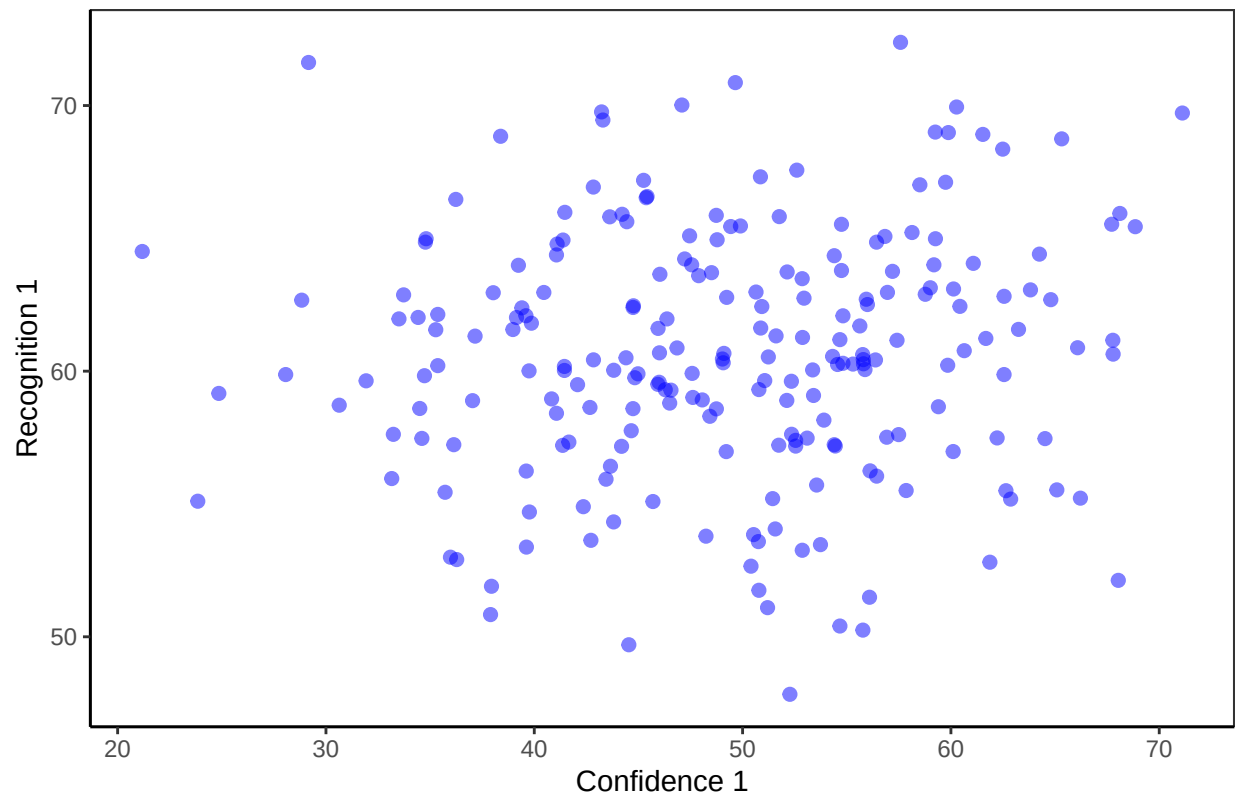


Make a scatterplot of confidence 1 (`conf1`) and recognition 1 (`rec1`) to indicate how their first ratings are related.

```
scatter_line = ggplot(nomissing_midterm) + geom_point(
  aes(x=conf1, y=rec1),
  color="blue",
  size=2,
  alpha=0.5
) +
scale_x_continuous(name="Confidence 1", breaks= seq(0, 80, 10)) +
scale_y_continuous(name="Recognition 1", breaks= seq(0, 100, 10)) +
ggtitle(label="Scatterplot of confidence 1 and recognition 1")

scatter_line + theme_bw() + tidyUp
```

Scatterplot of confidence 1 and recognition 1



Summary:

The scatter plot illustrates a positive relationship between the ratings for confidence 1 and recognition 1, suggesting as rec1 increases, the conf1 increases in most data points. The relationship is moderately strong.